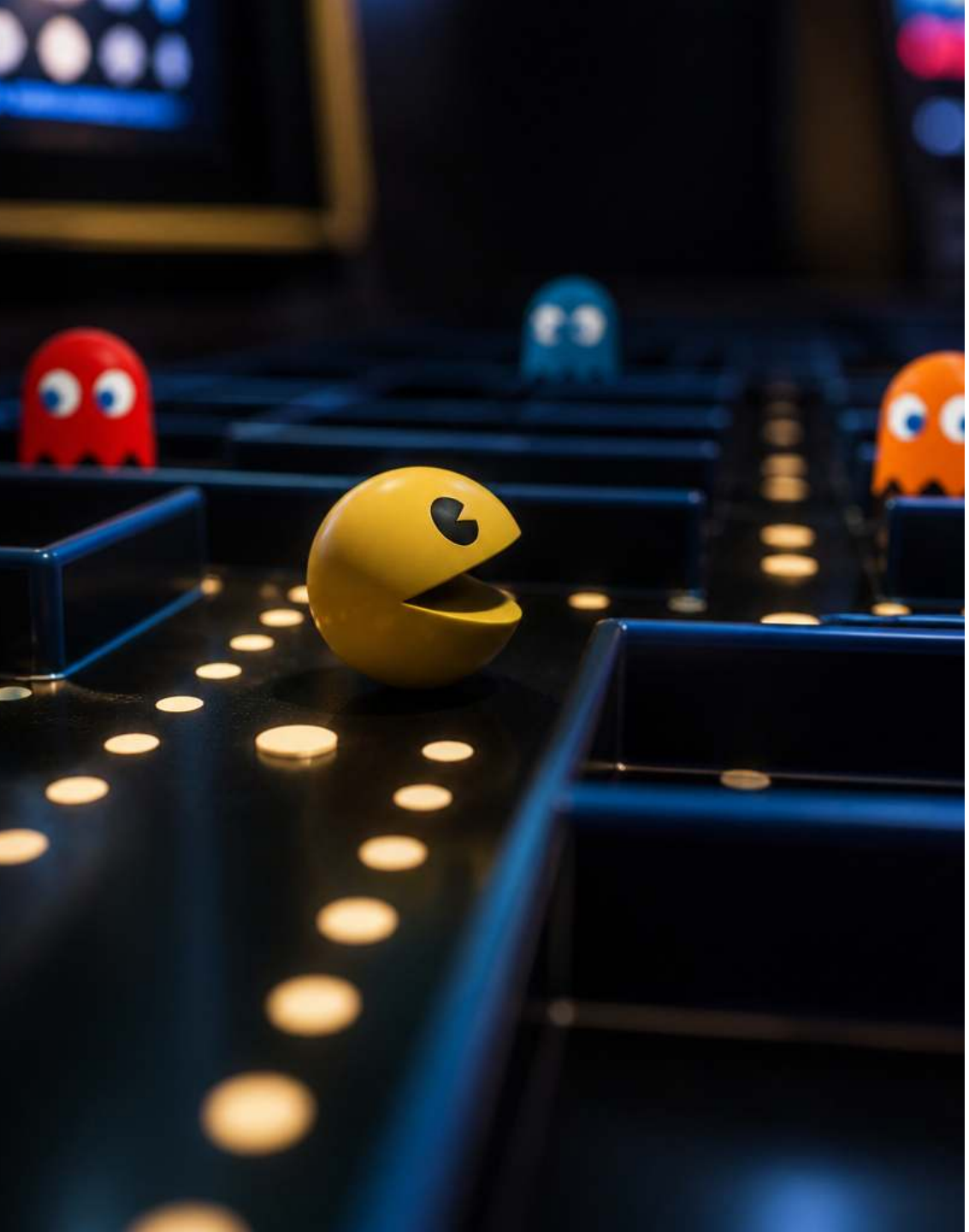


Pacman Game

Master 1 Artificial Intelligence

Group G

Abdelbaki Kacem
M1 AI

Abou Orm Daniel
M1 AI

Kabban Joe
M1 AI

UNIVERSITÉ
CÔTE D'AZUR

1 Maze Implementation

The different algos to generate the maze.

2 Project implementation

BackEnd + FrontEnd.

3 MongoDB Implementation

Tables in our Database.

4 Ghosts & AI

Exploring the artificial intelligence of our characters.

5 System Architecture

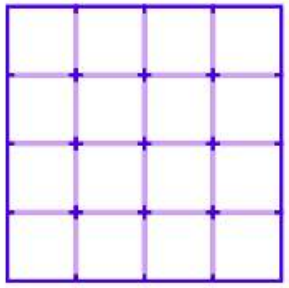
Understanding the underlying structure.

6 Our project features

Small demo.

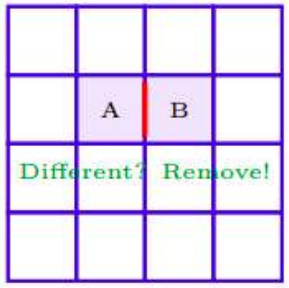
Kruskal's Algorithm

Step 1: Walls = Edges



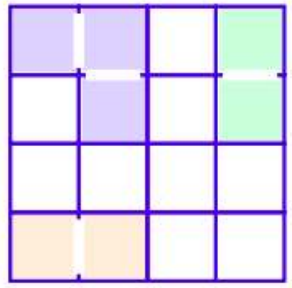
Each wall is an edge. Each cell is its own set (Union-Find).

Step 2: Pick Wall, Check Sets



Shuffle walls. Remove if cells are in different sets, then union.

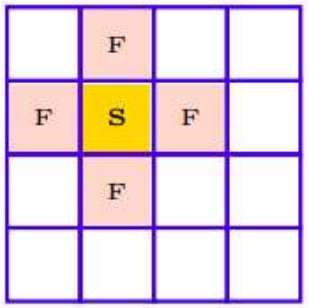
Step 3: Merge Sets



Colors = disjoint sets. Sets merge until all cells are connected.

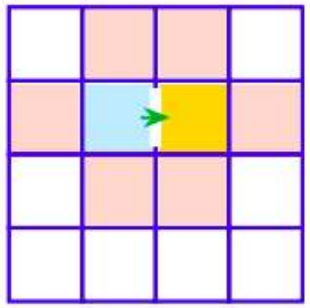
Prim's Algorithm

Step 1: Start + Frontier



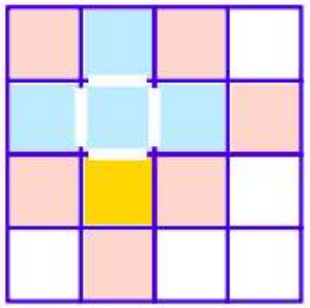
Start in maze. Neighbours = frontier (coral).

Step 2: Pick Frontier Cell



Random frontier cell joins maze. Frontier expands.

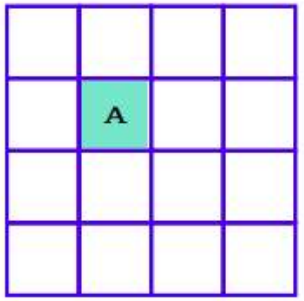
Step 3: Grows Outward



Blob-like growth. Shorter corridors, more branching than DFS.

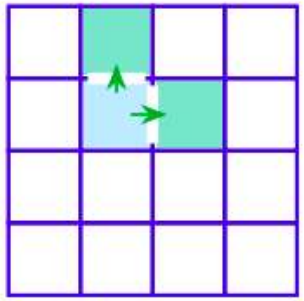
Tetris (Active Cells)

Step 1: One Active Cell



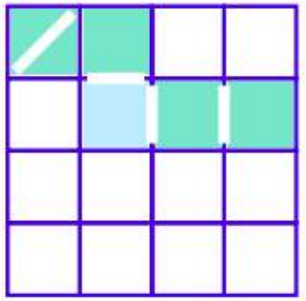
Active list = {A}. Active cells can still grow.

Step 2: Branch Bias 85%



Prefer cells with ≥2 free neighbours.

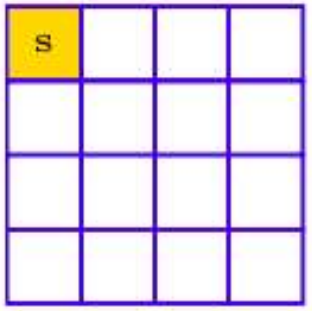
Step 3: Both Stay Active



Parent + child both re-enter active list → many fronts.

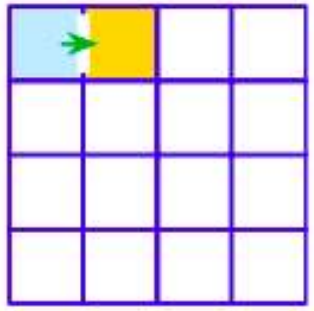
DFS (Depth-First Search)

Step 1: Init Grid



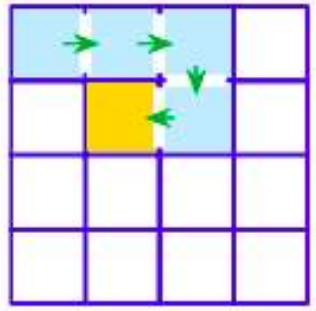
All walls up. Random start.

Step 2: Carve



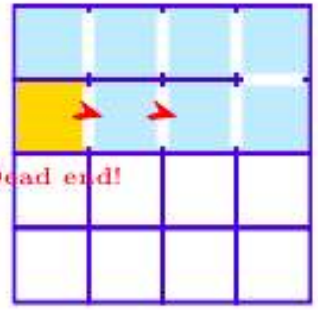
Remove wall, move to neighbour.

Step 3: Go Deep



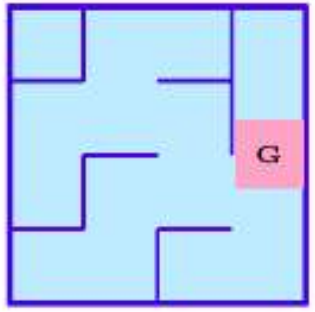
Always go deeper first.

Step 4: Backtrack



No free neighbours → backtrack.

Step 5: Half-Maze Done



Perfect maze on half-grid.

Step 6: Cycles + Mirror + Seal

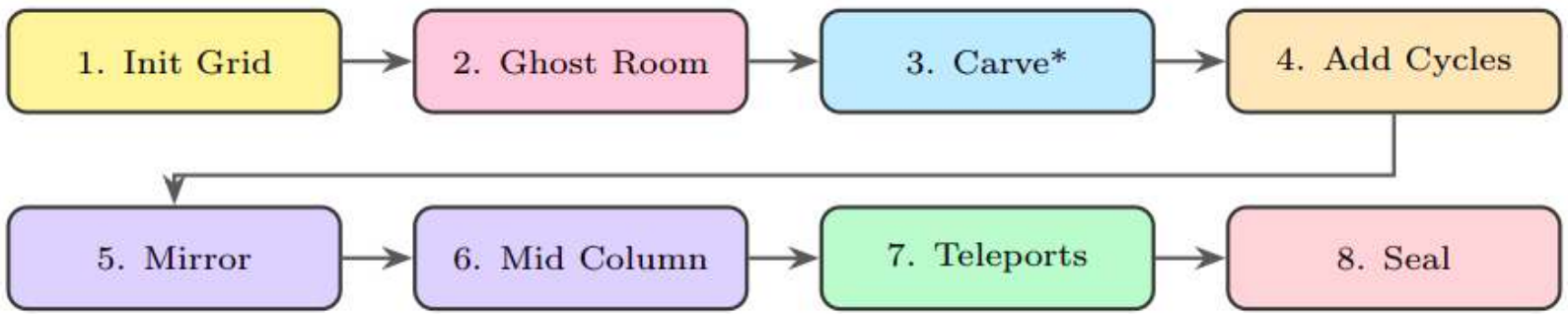


Add loops, mirror, teleports, seal.

Algorithm Comparison

	DFS	Kruskal	Prim	Tetris
Strategy	Recursive stack	Random edges	Frontier growth	Active cells
Corridors	Long, winding	Medium, uniform	Short, branchy	Short, dense
Branching	Low	Medium	High	Very high

All algorithms share the same post-processing: mirror, ghost room, teleports, and sealed borders.(Inheriting from Parent class)

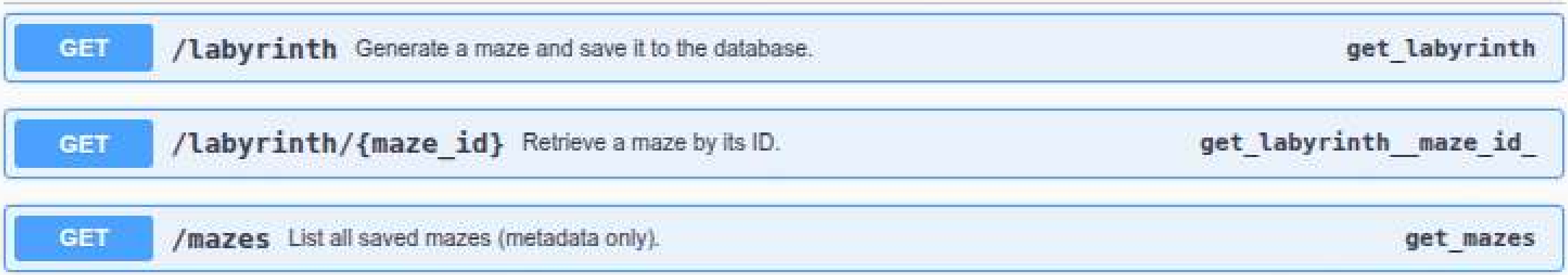


**Step 3 uses DFS, Kruskal, Prim, or Tetris. All other steps are shared.*

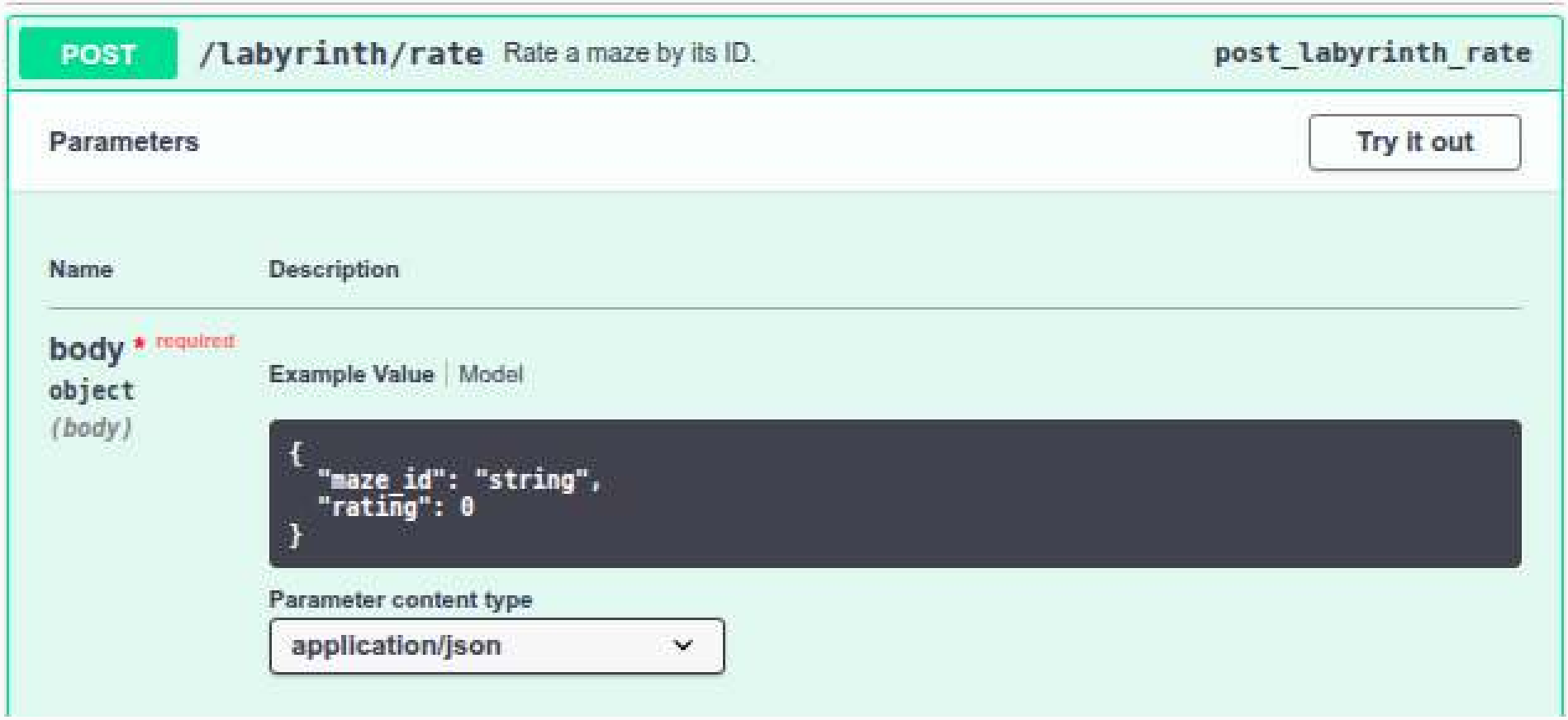
BACK END

Swagger's Labyrinth APIs

Maze generation and retrieval



Maze rating


POST
/labyrinth/rate
Rate a maze by its ID.
post_labyrinth_rate
Parameters
Try it out
Name
Description
body
required
object
(body)
Example Value
Model
{
"maze_id": "string",
"rating": 0
}
Parameter content type
application/json

Game replay management

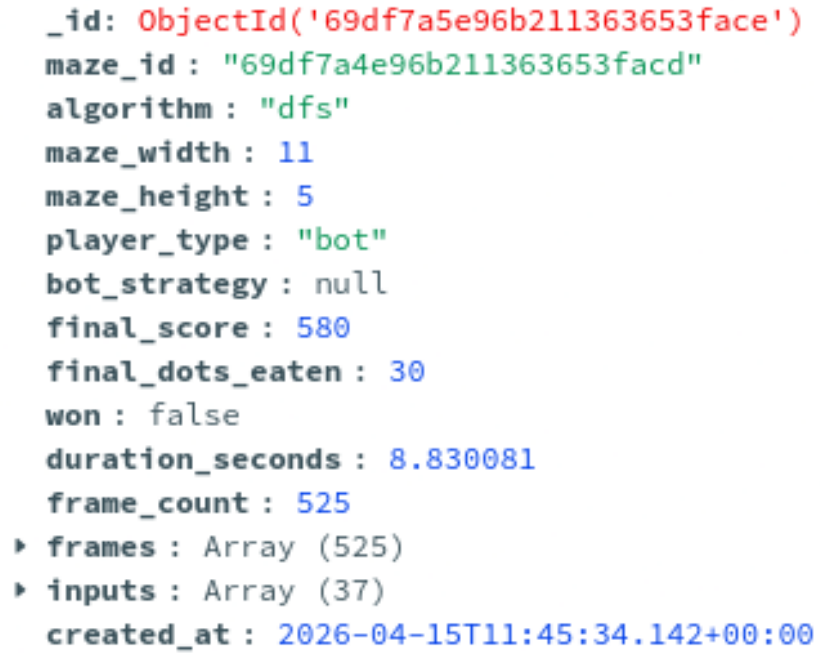
POST	/replay	Save a game replay.	post_replay
GET	/replay/{replay_id}	Retrieve a replay by its ID.	get_replay__replay_id_
GET	/replays	List saved replays (metadata only).	get_replays

Maze table entry



```
_id: ObjectId('69cf7b5e87e383c81985c9a9')
algorithm : "dfs"
height : 5
width : 6
▸ maze : Object
rating : 2
rating_count : 1
rating_sum : 2
created_at : 2026-04-03T08:33:34.957+00:00
```

Replay table entry



```
_id: ObjectId('69df7a5e96b211363653face')
maze_id : "69df7a4e96b211363653facd"
algorithm : "dfs"
maze_width : 11
maze_height : 5
player_type : "bot"
bot_strategy : null
final_score : 580
final_dots_eaten : 30
won : false
duration_seconds : 8.830081
frame_count : 525
▸ frames : Array (525)
▸ inputs : Array (37)
created_at : 2026-04-15T11:45:34.142+00:00
```

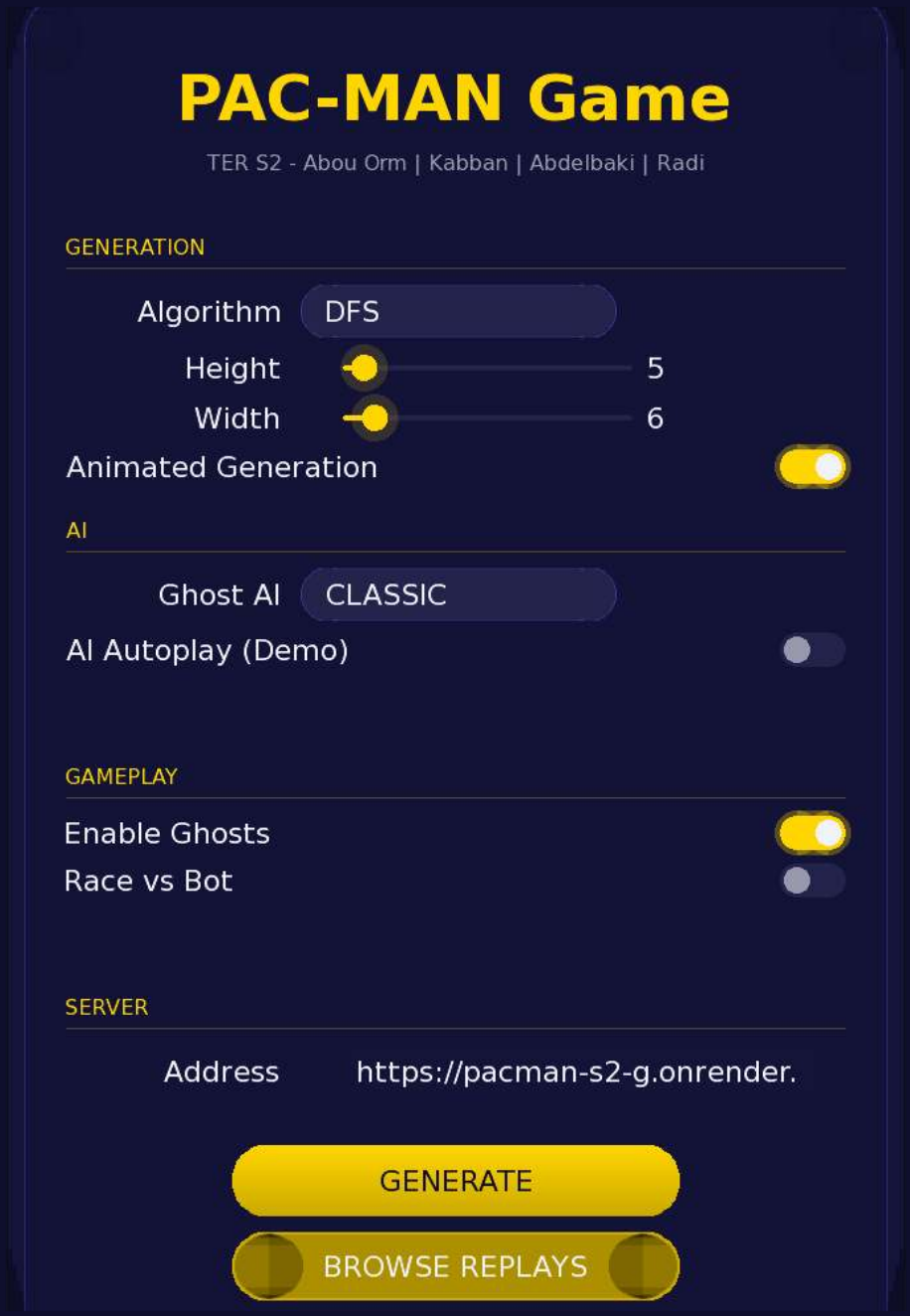
MongoDB Data Model & Architecture

Collections

- **`mazes` Collection**
  - Algorithm used (e.g., DFS, Prim's)
  - Dimensions (width, height)
  - Full maze structure data
  - Community rating system (average, count, sum)
- **`replays` Collection**
  - Player type (human/bot)
  - Final score
  - Dots eaten, win/loss status
  - Game duration
  - Frame-by-frame game state and player inputs

FRONT-END

MENU SCREEN


PAC-MAN Game
TER S2 - Abou Orm | Kabban | Abdelbaki | Radi
GENERATION
Algorithm
DFS
Height
5
Width
6
Animated Generation
AI
Ghost AI
CLASSIC
AI Autoplay (Demo)
GAMEPLAY
Enable Ghosts
Race vs Bot
SERVER
Address
https://pacman-s2-g.onrender.
GENERATE
BROWSE REPLAYS

Browse Replays menu



DEMO-REPLAY


< Back
SAVED REPLAYS
Found 33 replay(s).
WATCH
DFS (11x5) - HUMAN
WATCH
DFS (11x5) - HUMAN
WATCH
DFS (11x5) - BOT
WATCH
DFS (11x5) - BOT
Score: 2370 | Result: WON | 2020-04-15 11:57:11.912000
WATCH
DFS (11x5) - BOT
WATCH
DFS (11x5) - BOT
Score: 770 | Result: WON | 2020-04-09 19:29:32.037000
WATCH
DFS (11x5) - BOT
Score: 770 | Result: WON | 2020-04-08 21:11:32.851000
WATCH
DFS (11x5) - BOT
Score: 1970 | Result: WON | 2020-04-08 21:10:53.555000
WATCH
DFS (11x5) - HUMAN
WATCH
DFS (31x11) - HUMAN
WATCH
DFS (21x15) - HUMAN
WATCH

Ghost AI: Modes of Operation



SCATTER

Ghosts momentarily retreat to their designated corner of the maze, offering a brief respite before returning to the chase.



CHASE

Ghosts actively pursue Pac-Man using their unique targeting strategies. This is the primary offensive state.



FRIGHTENED

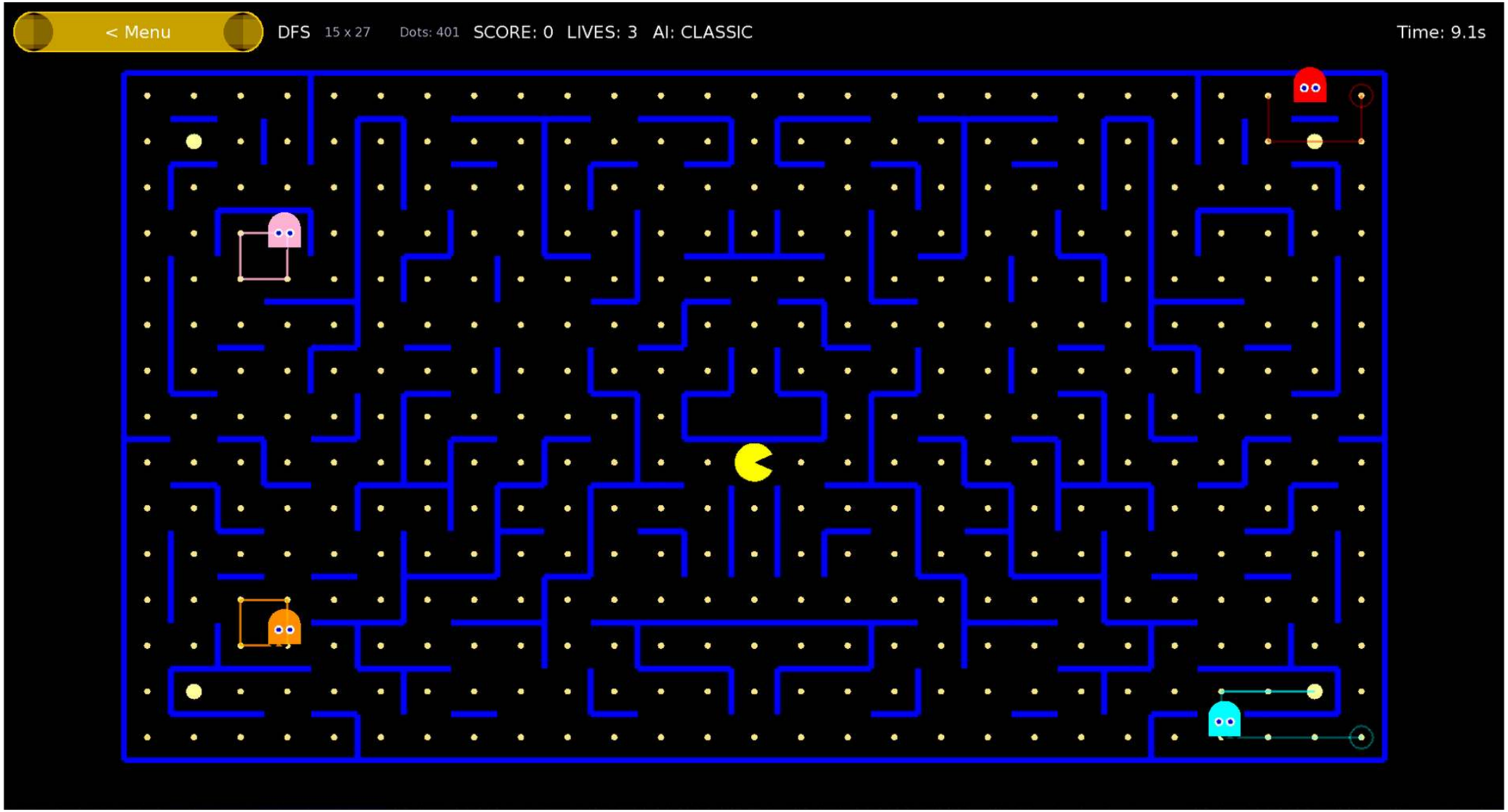
Triggered by a Power Pellet. Ghosts turn blue, move randomly, and attempt to evade Pac-Man. Vulnerable to being eaten.



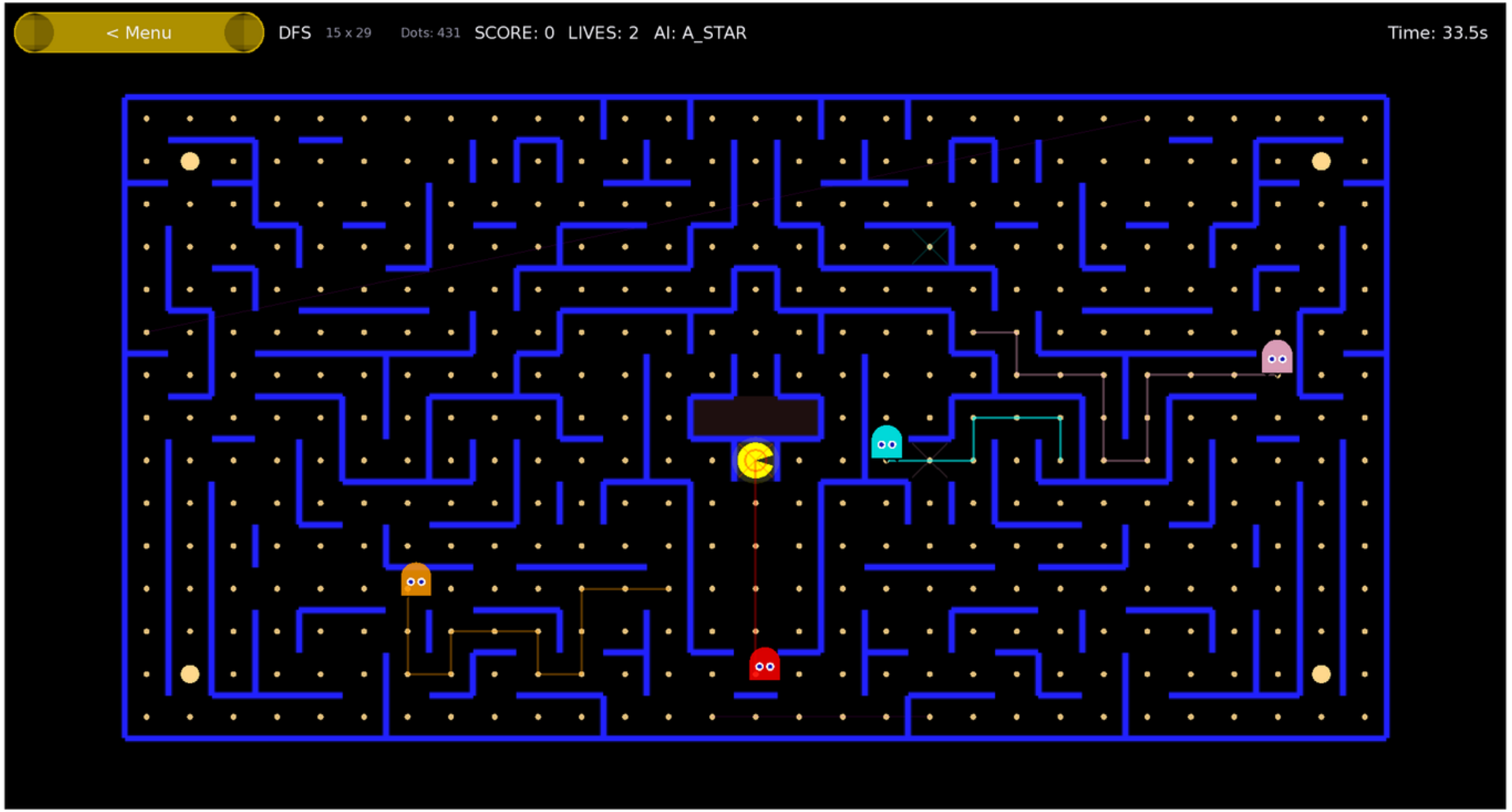
EATEN

After being consumed by Pac-Man, ghosts return to the ghost pen as eyes, passing through walls to respawn.

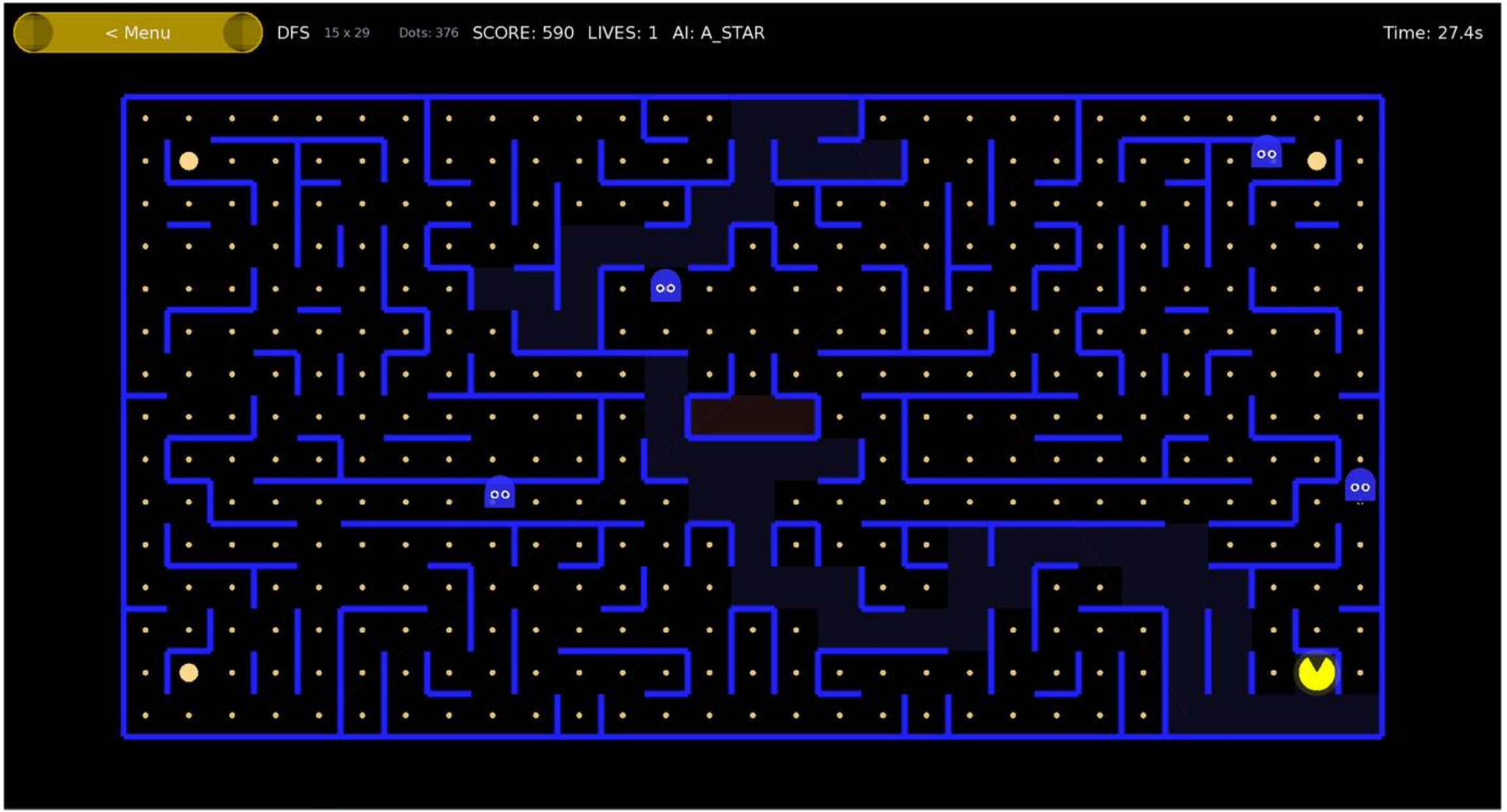
Scatter mode(*Visiting corners*)



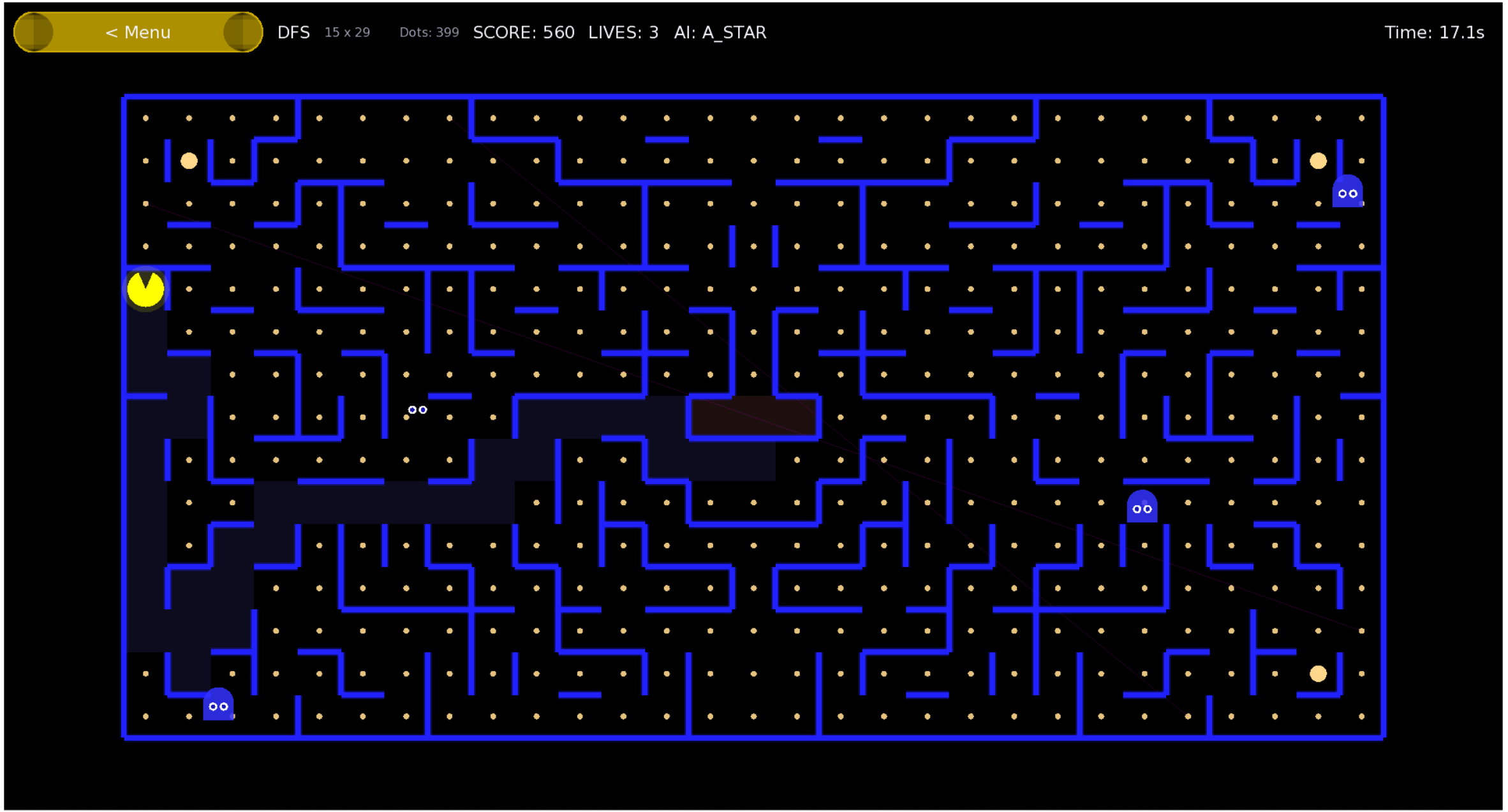
Chase mode(*Targeting pacman*)



Frightened mode(*escaping pacman*)



Eaten mode(*Going back to base*)



The Four Apparitions

Individual Ghost Behaviors in Pac-Man

Blinky (Red)



Directly targets Pac-Man's position (Aggressive).

Pinky (Pink)



Targets a position 4 tiles ahead of Pac-Man (Ambusher).

Inky (Cyan)



Target position is calculated based on Blinky and Pac-Man (Unpredictable).

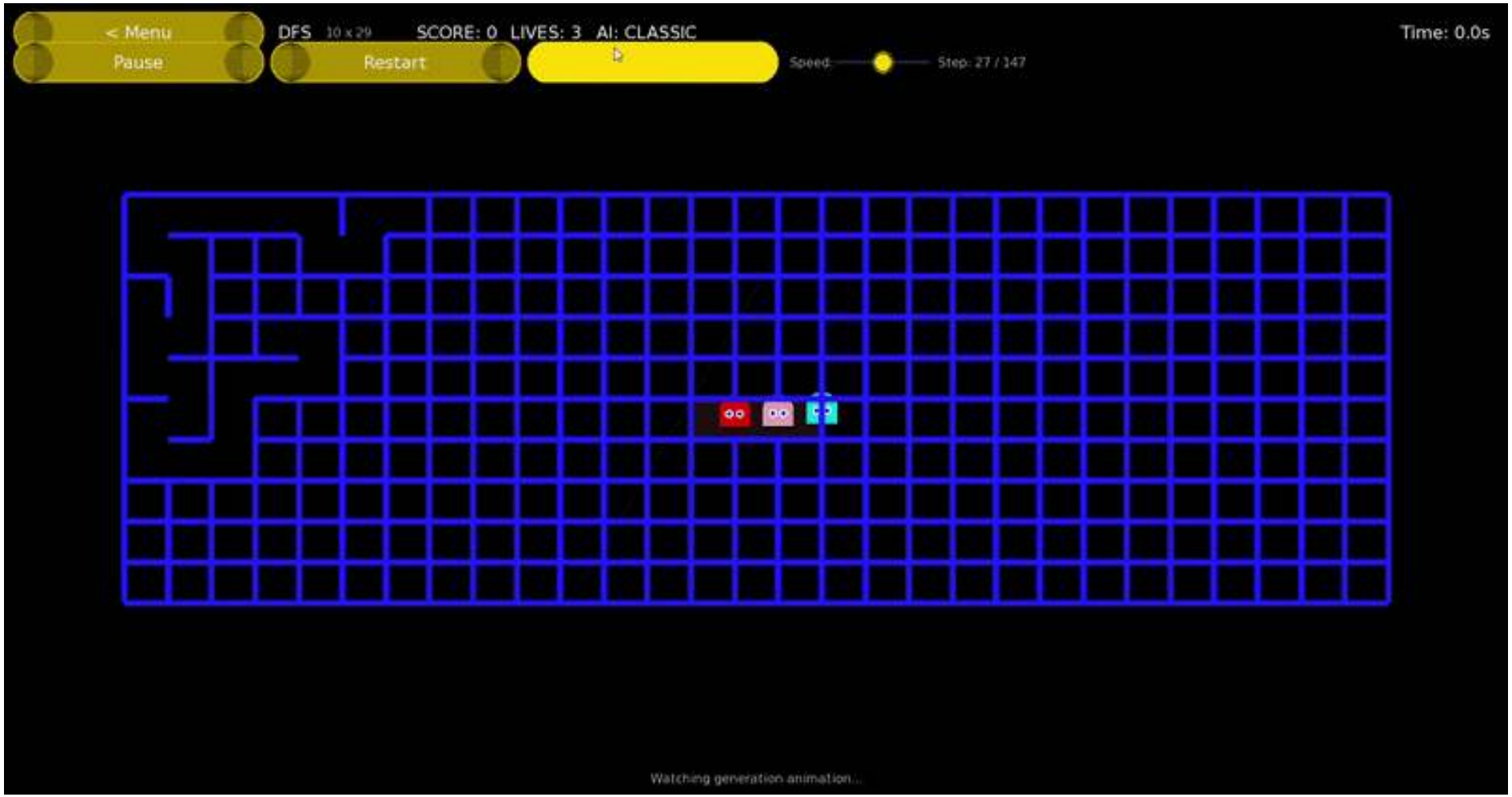
Clyde (Orange)



Chases when far, scatters when close (Timid).

Blinky



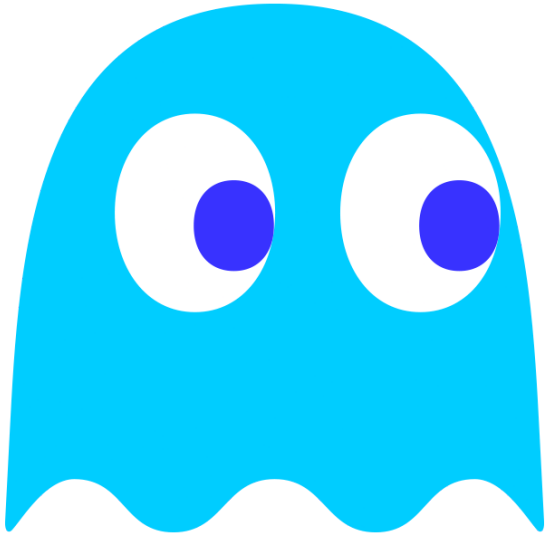

< Menu
DFS 10 x 29
SCORE: 0 LIVES: 3 AI: CLASSIC
Time: 0.0s
Pause
Restart
Speed:
Step: 27 / 147
Watching generation animation...

Pinky




< Menu
SCORE: 0 LIVES: 3 AI: CLASSIC
Time: 0.0s
Loading maze...
WASD/Arrows: Move | Scroll: Zoom | G: Toggle Ghost AI (A*/Classic) | ESC: Menu

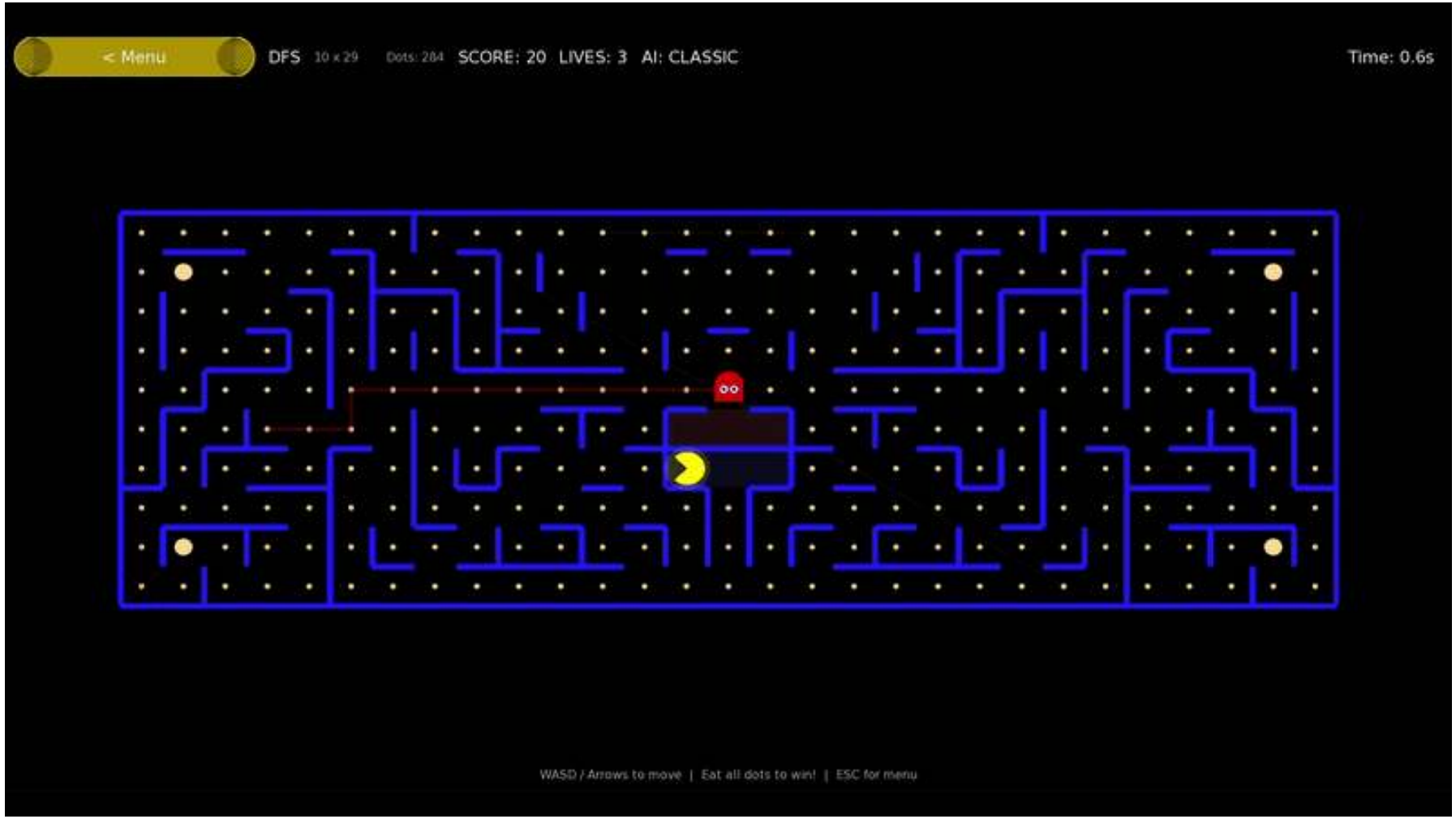
Inky




< Menu
DFS 10 x 29 Dots: 95 SCORE: 2030 LIVES: 2 AI: CLASSIC
Time: 46.7s
WASD / Arrows to move | Eat all dots to win! | ESC for menu

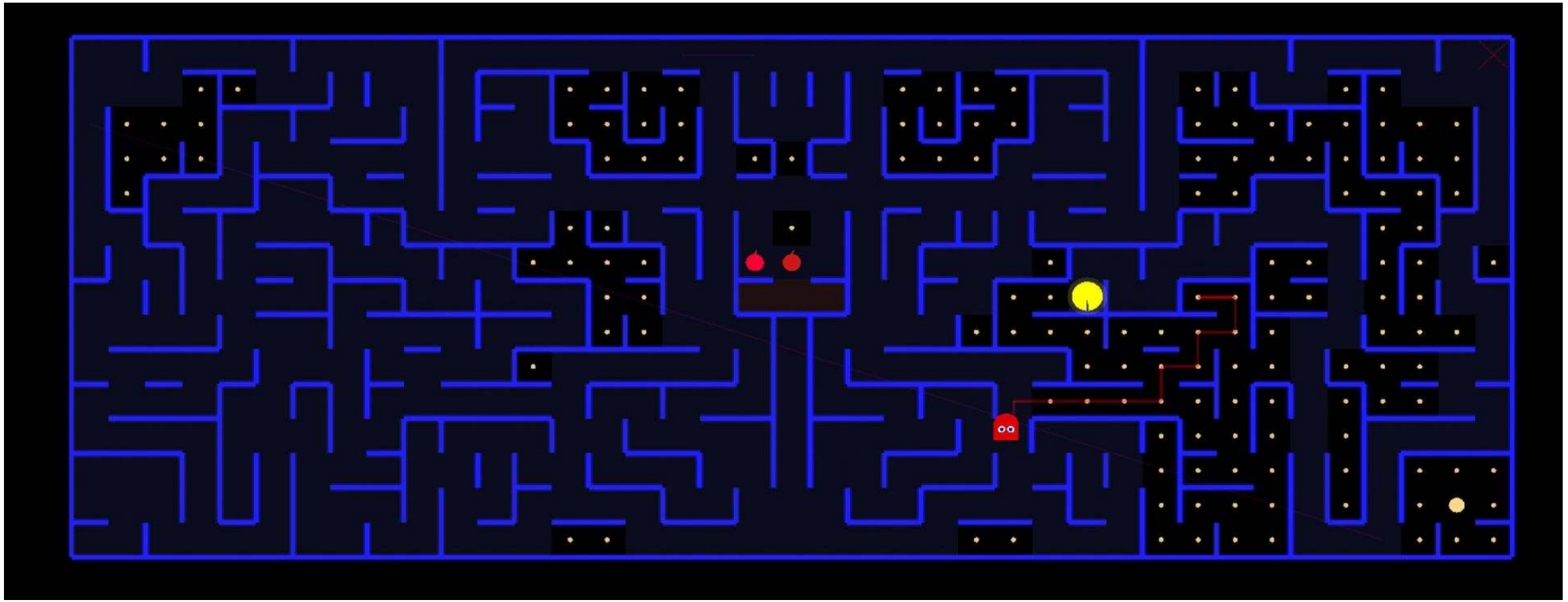
Clyde




< Menu
DFS 10 x 29 Dots: 284 SCORE: 20 LIVES: 3 AI: CLASSIC
Time: 0.6s
WASD / Arrows to move | Eat all dots to win! | ESC for menu

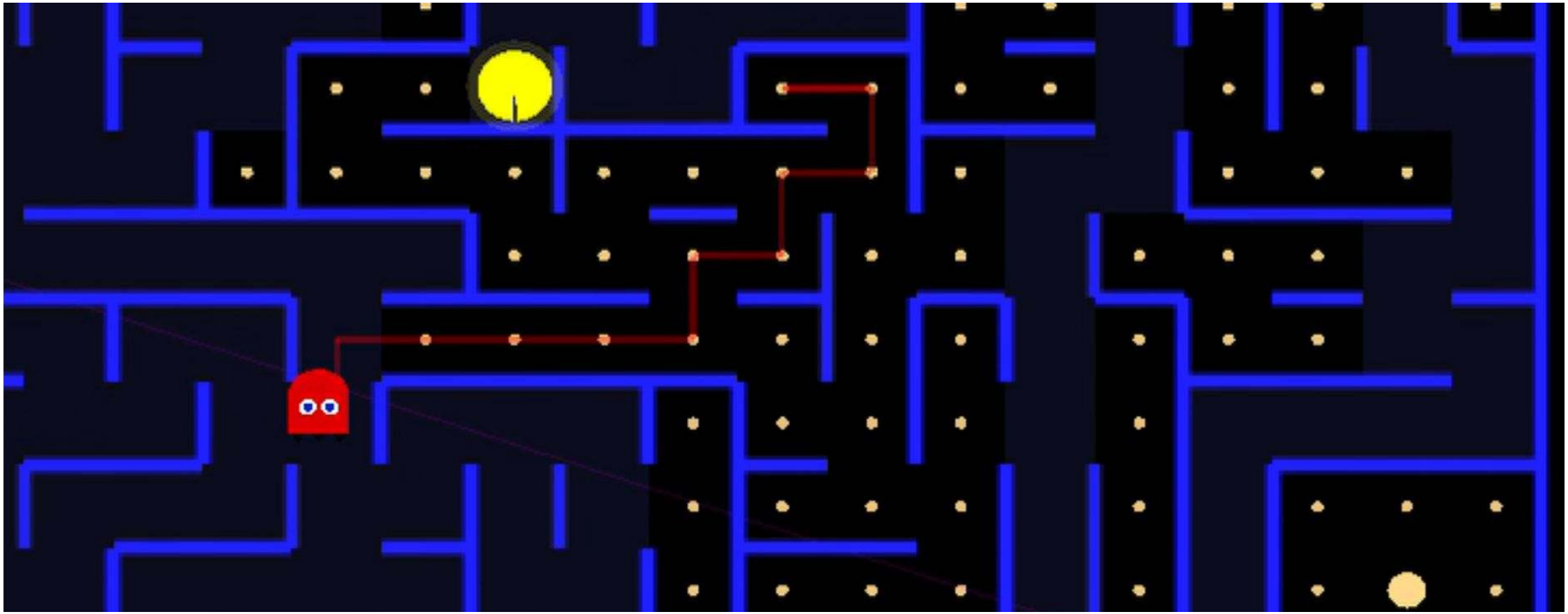
Ghost AI: Difficulty Levels

Comparing Pursuit and Navigation Strategies



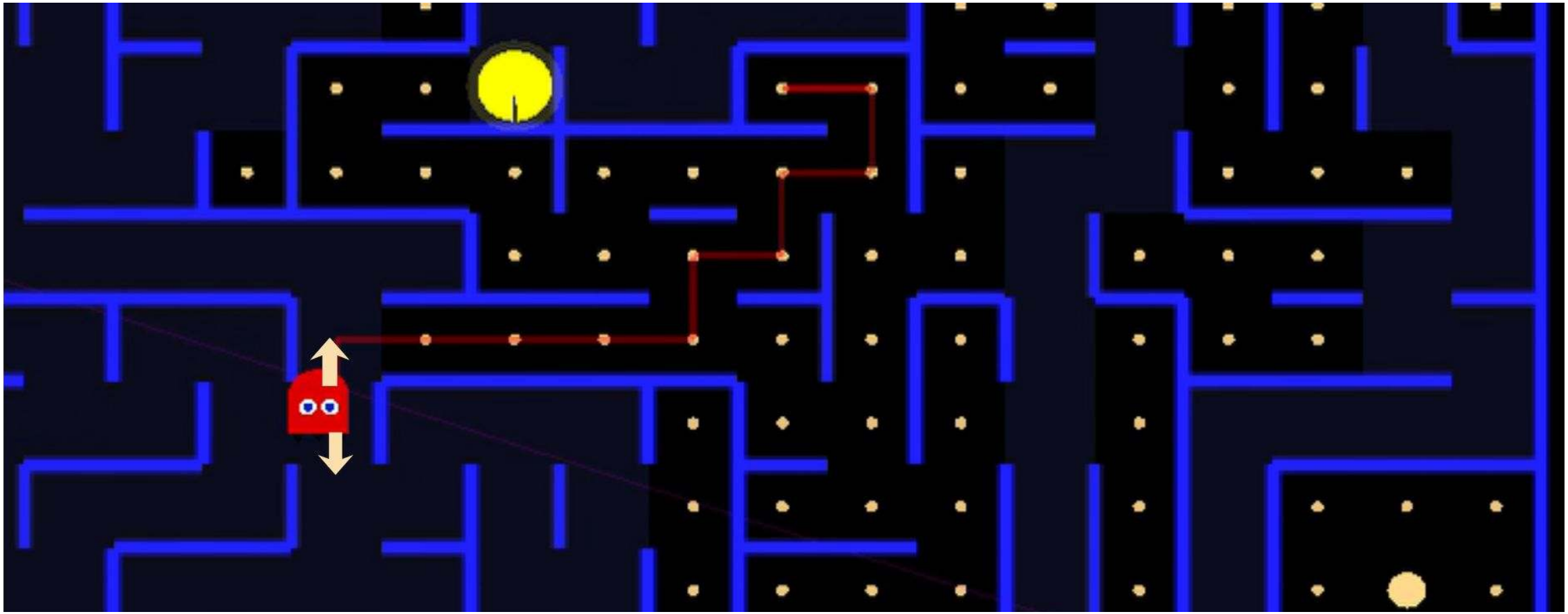
Ghost AI: Difficulty Levels

Greedy



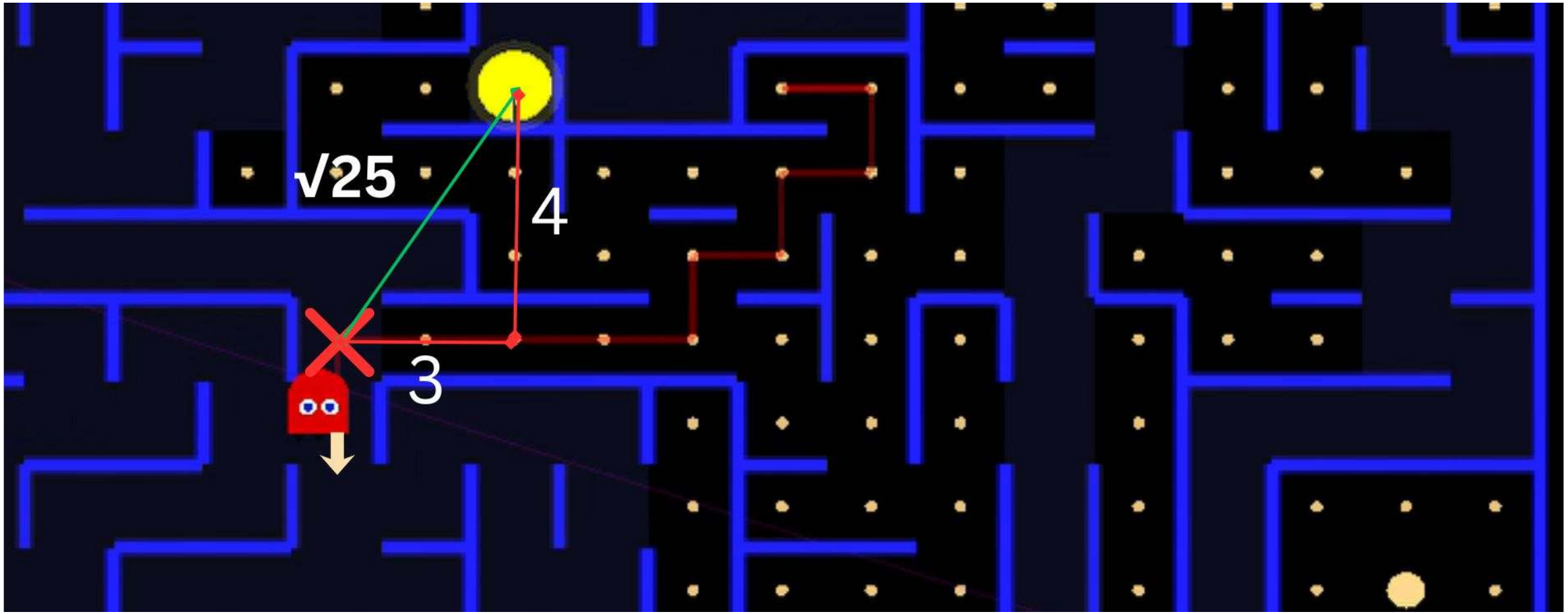
Ghost AI: Difficulty Levels

Greedy



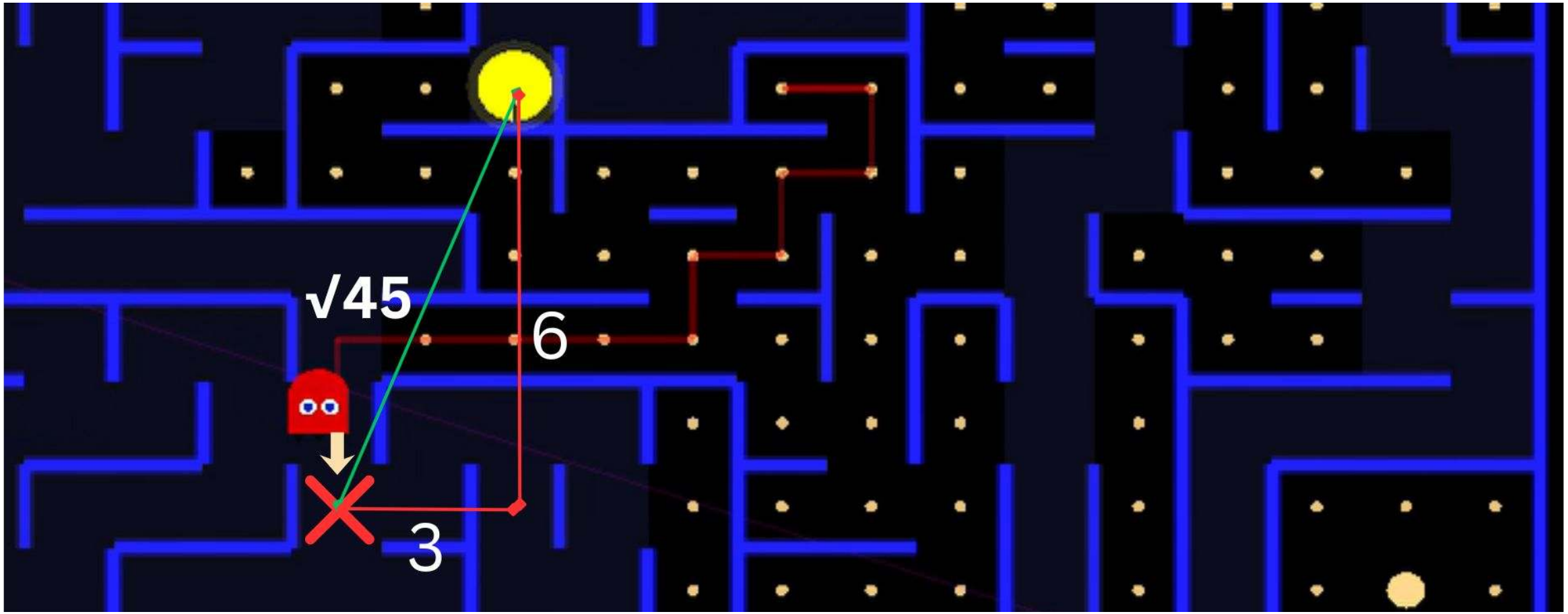
Ghost AI: Difficulty Levels

Greedy



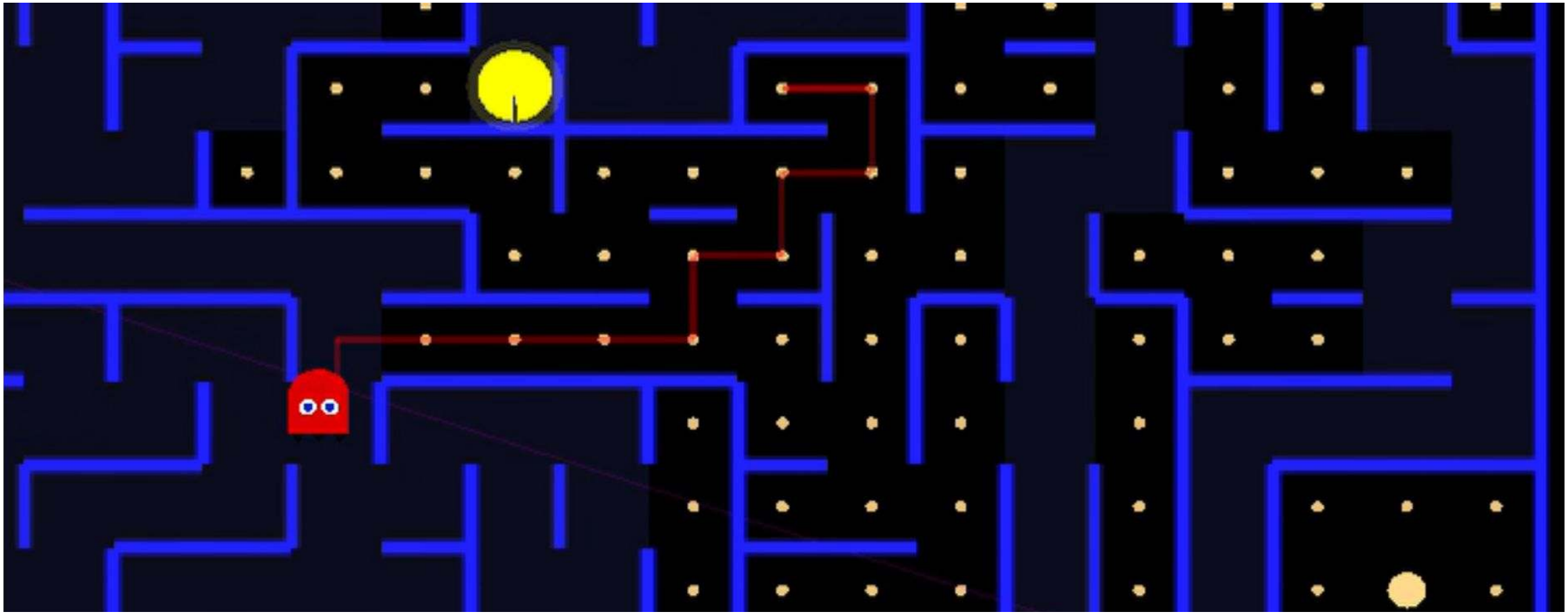
Ghost AI: Difficulty Levels

Greedy



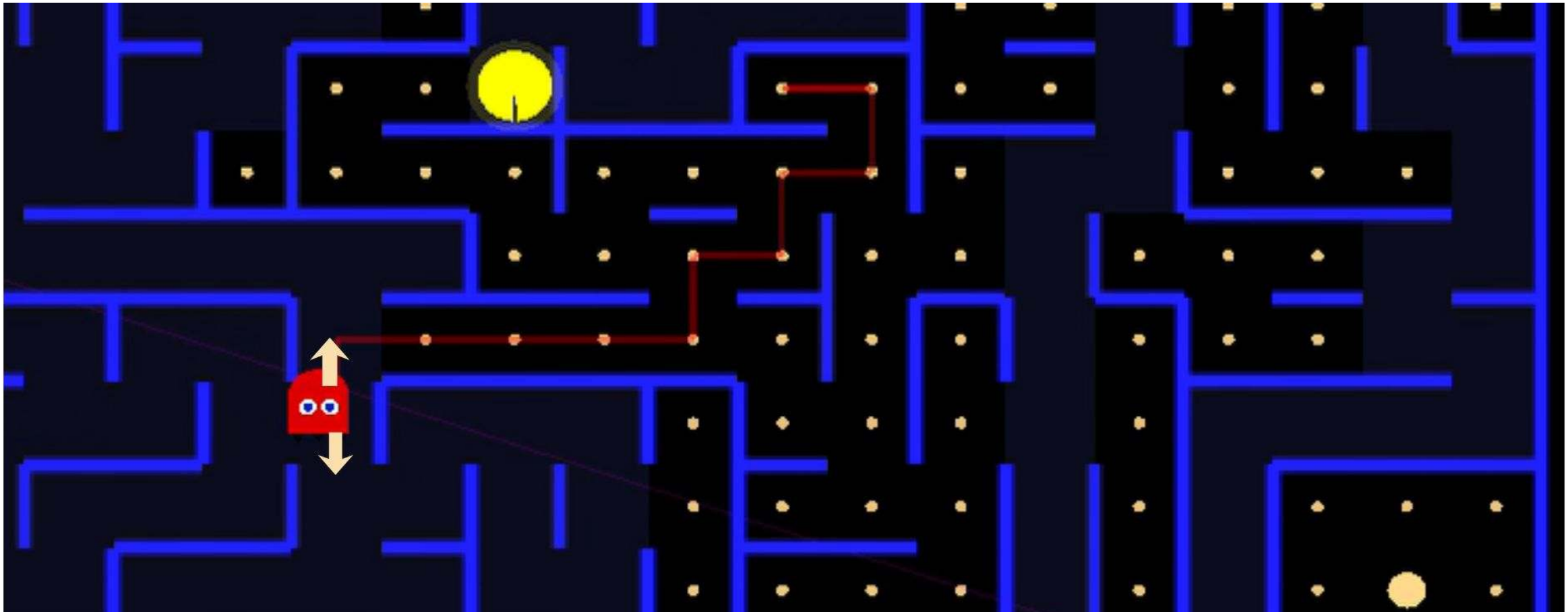
Ghost AI: Difficulty Levels

A Star



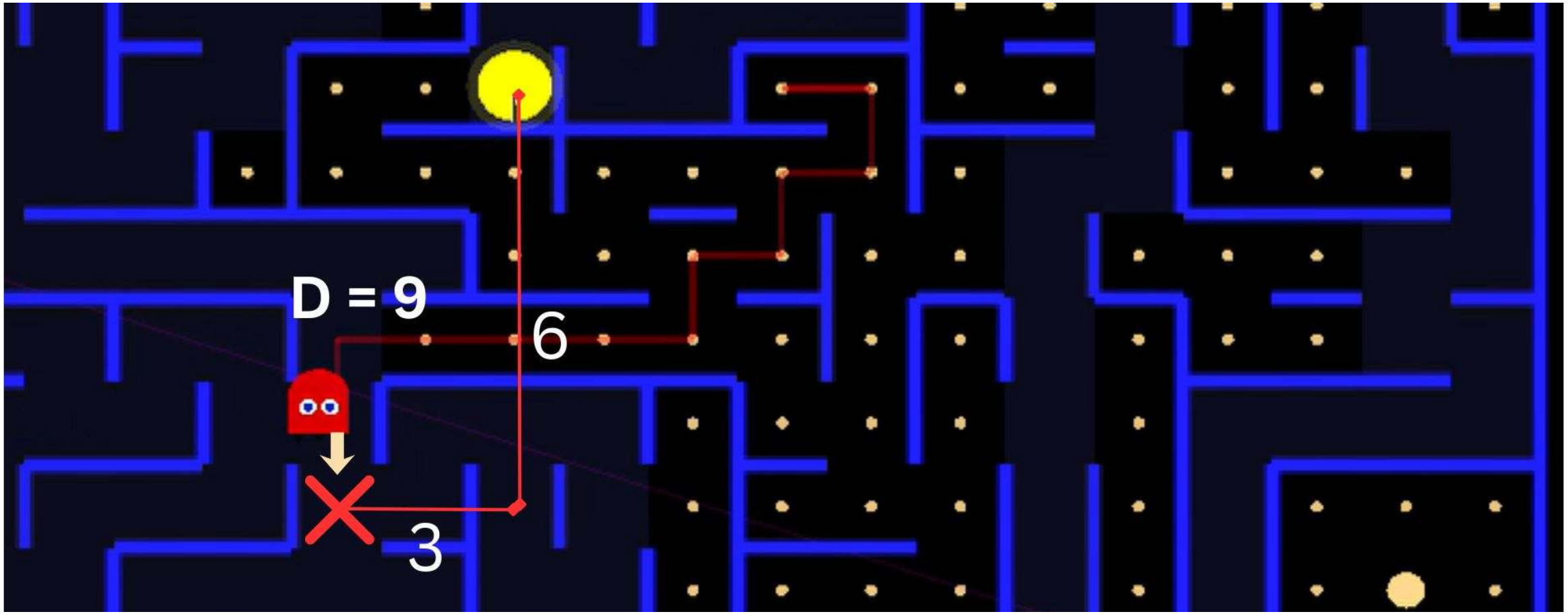
Ghost AI: Difficulty Levels

A Star



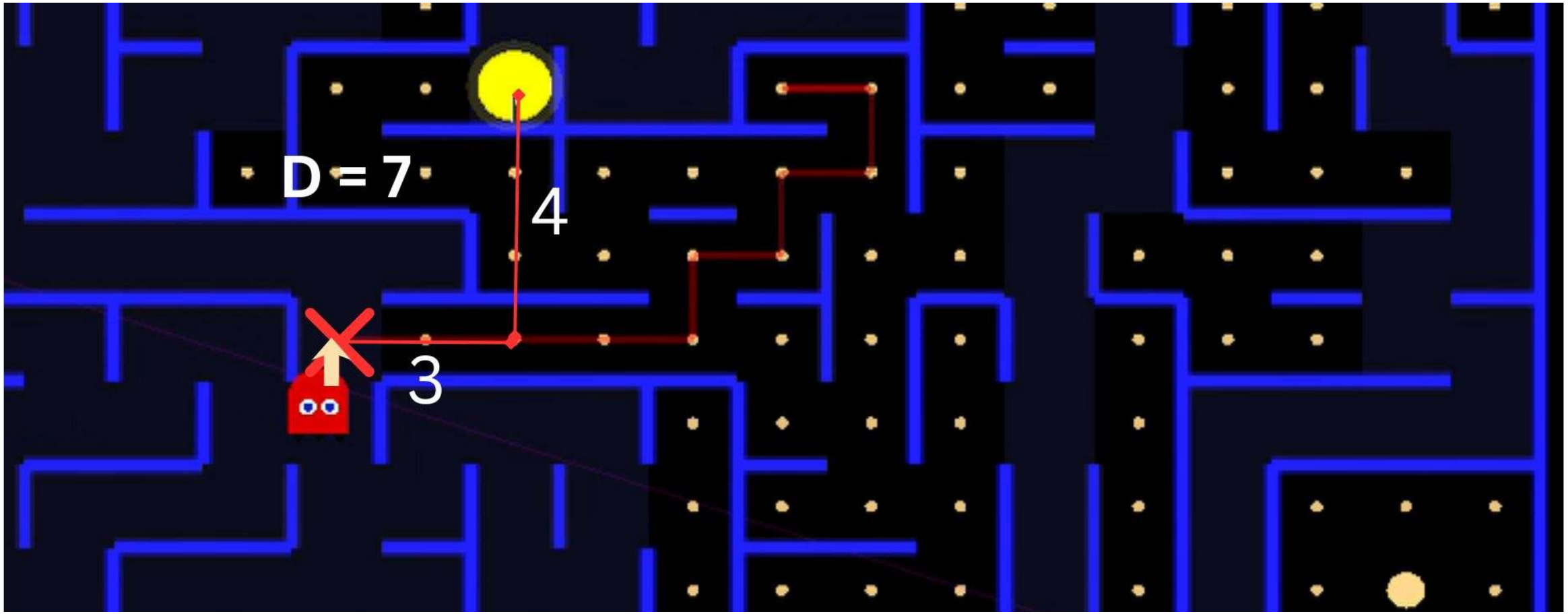
Ghost AI: Difficulty Levels

A Star



Ghost AI: Difficulty Levels

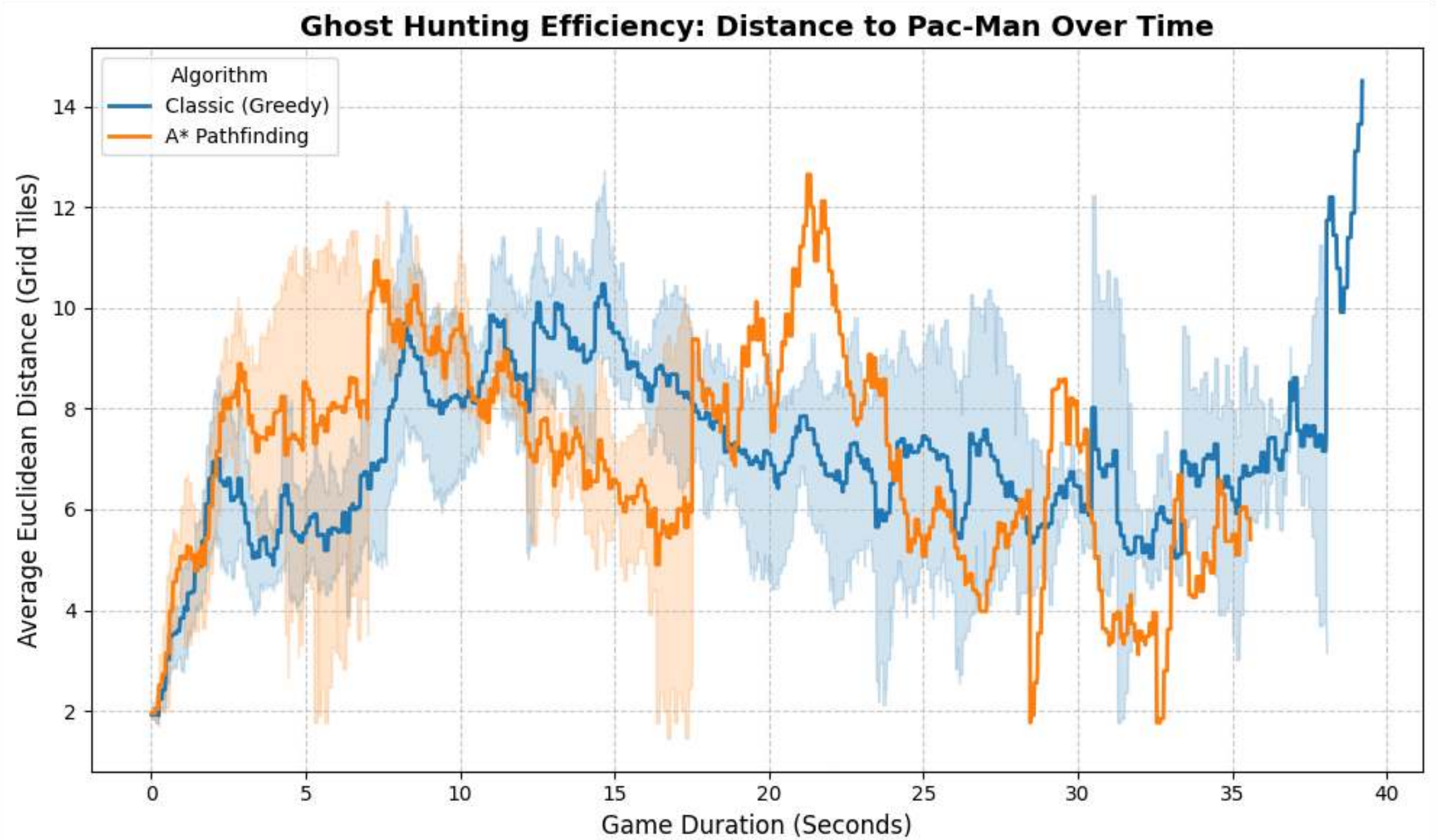
A Star



Ghost AI: Difficulty Levels

Comparing Pursuit and Navigation Strategies

Feature	Greedy	A* (A-Star)
Algorithm	Greedy Best-First Search	A* Search (Optimal Pathfinding)
Heuristic/Metric	Squared Euclidean Distance	Manhattan Distance (Heuristic)
Exploration	Evaluates immediate neighbors only	Explores the complete maze graph
Data Structure	Directional priority (Up; Left; Down; Right)	Priority Queue for exploration
Behavior	Fast but potentially suboptimal	Finds the most optimal path


Ghost Hunting Efficiency: Distance to Pac-Man Over Time
Algorithm
Classic (Greedy)
A* Pathfinding
Average Euclidean Distance (Grid Tiles)
Game Duration (Seconds)
2
4
6
8
10
12
14
0
5
10
15
20
25
30
35
40

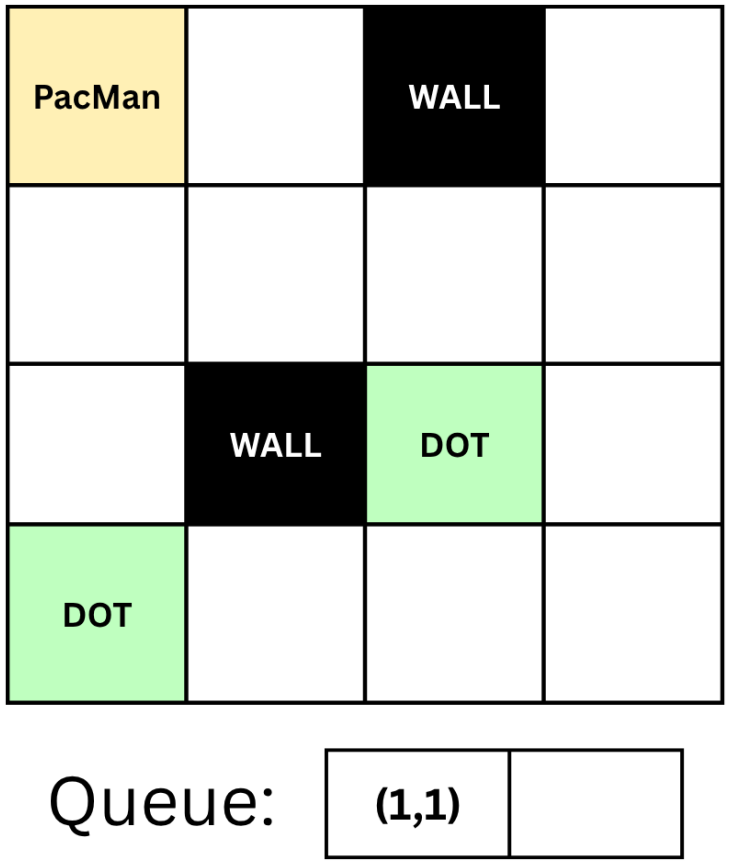
Player AI Bot Strategies

- **GREEDY Bot**
  - Utilizes Breadth-First Search (BFS) to locate the closest dot.
  - Prioritizes speed but disregarding ghost proximity.

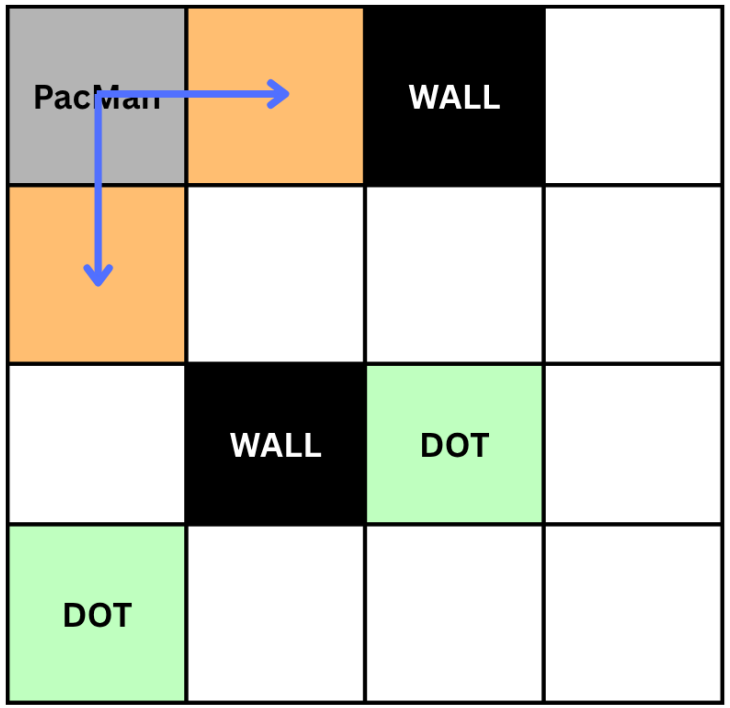
- **GHOST_AVOIDANCE Bot**
  - Employs a weighted Dijkstra algorithm.
  - Increases cell costs near ghosts to balance dot collection with evasion.



PacMan AI: Greedy (BFS)



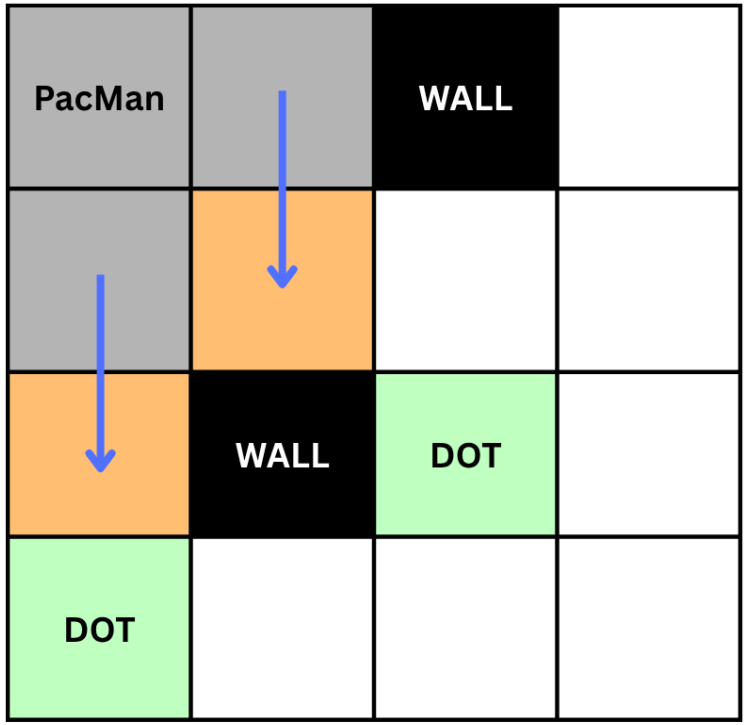
PacMan AI: Greedy (BFS)



Queue:

(1,2)	(2,1)

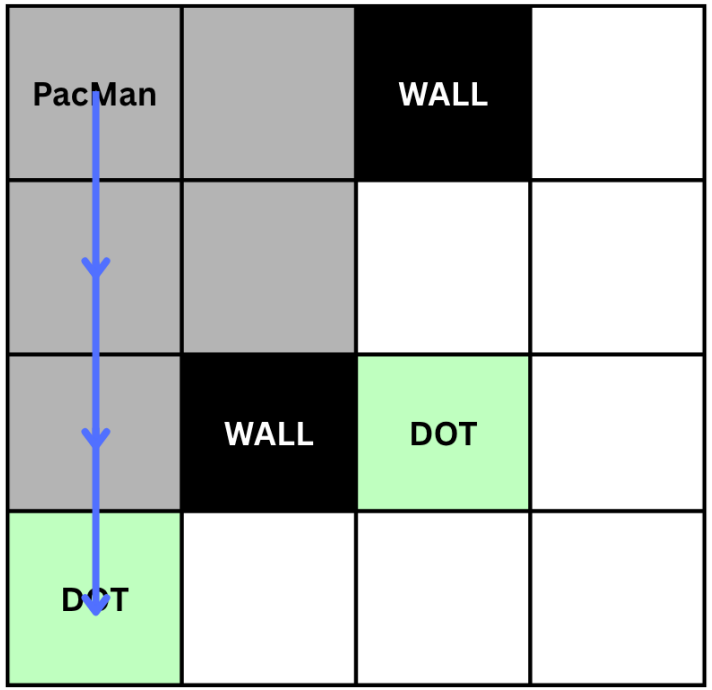
PacMan AI: Greedy (BFS)



Queue:

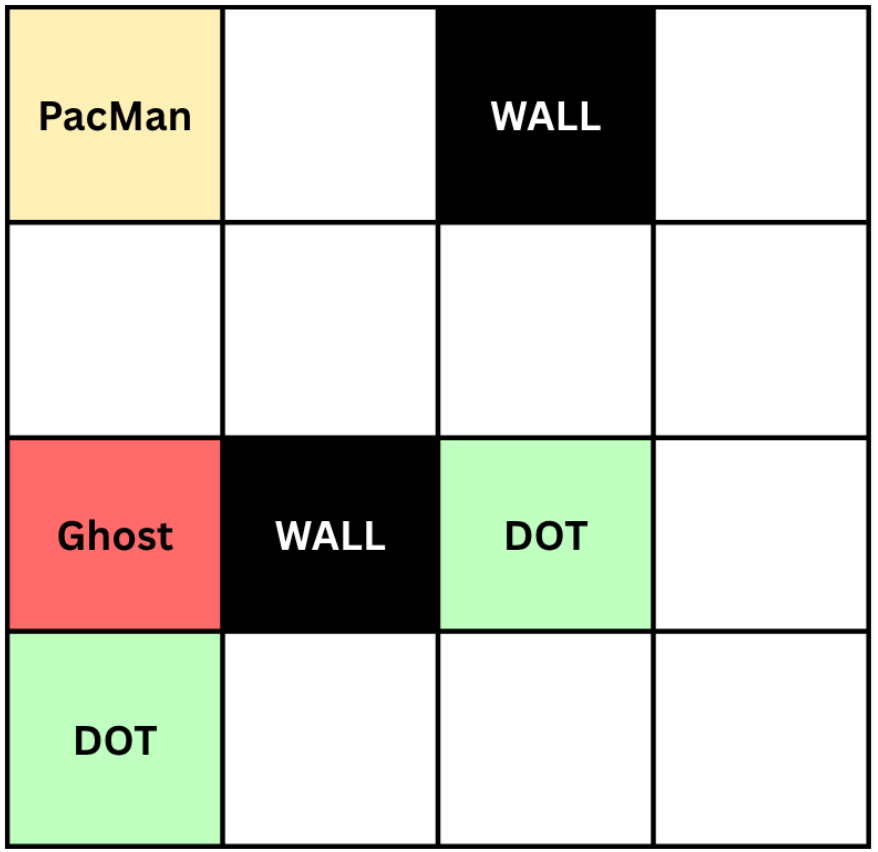
(3,1)	(2,2)

PacMan AI: Greedy (BFS)

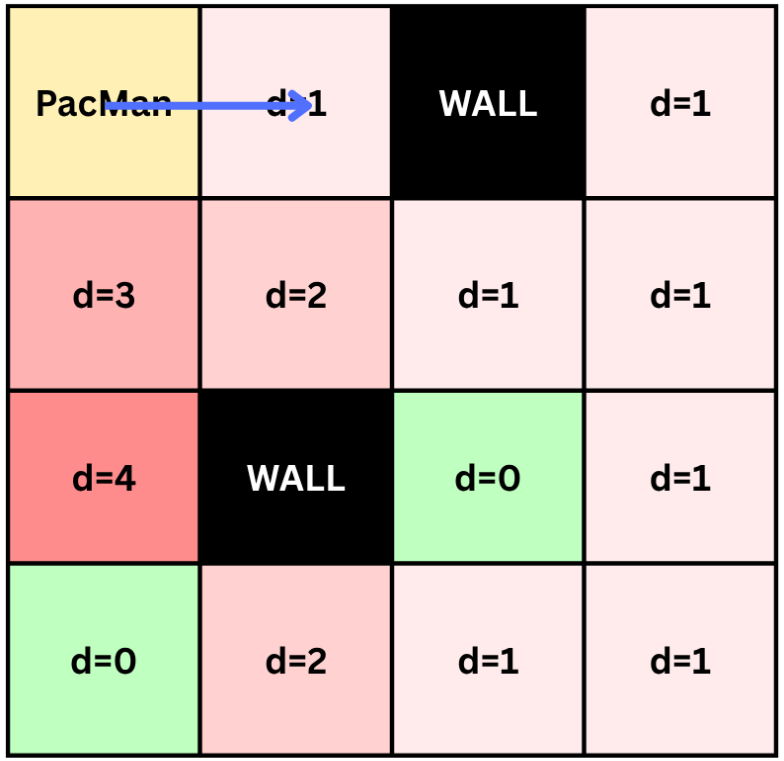


Path: (1,1)→(2,1)→(3,1) →(4,1)

PacMan AI: Ghost Avoidance (Weighted Dijkstra)

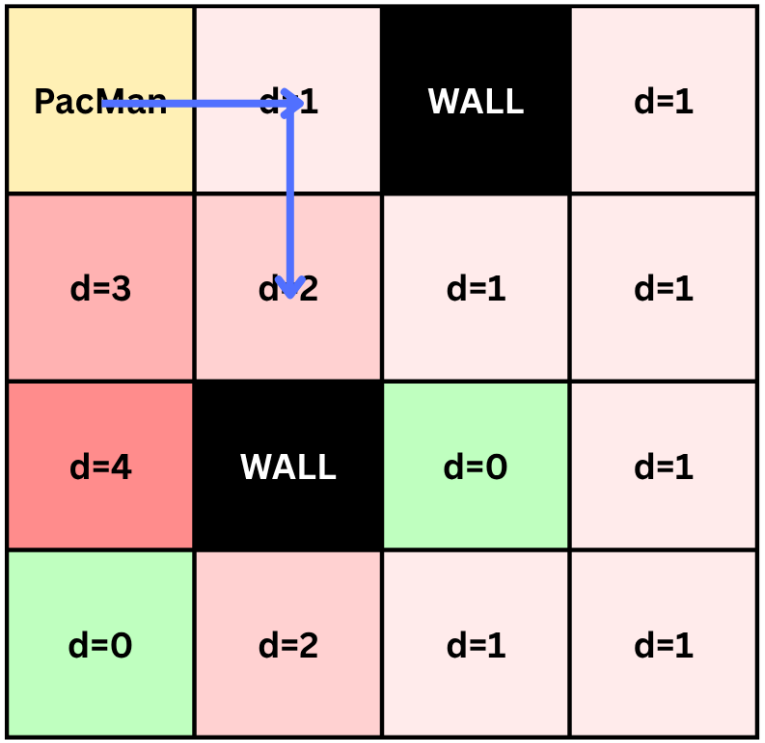


PacMan AI: Ghost Avoidance (Weighted Dijkstra)



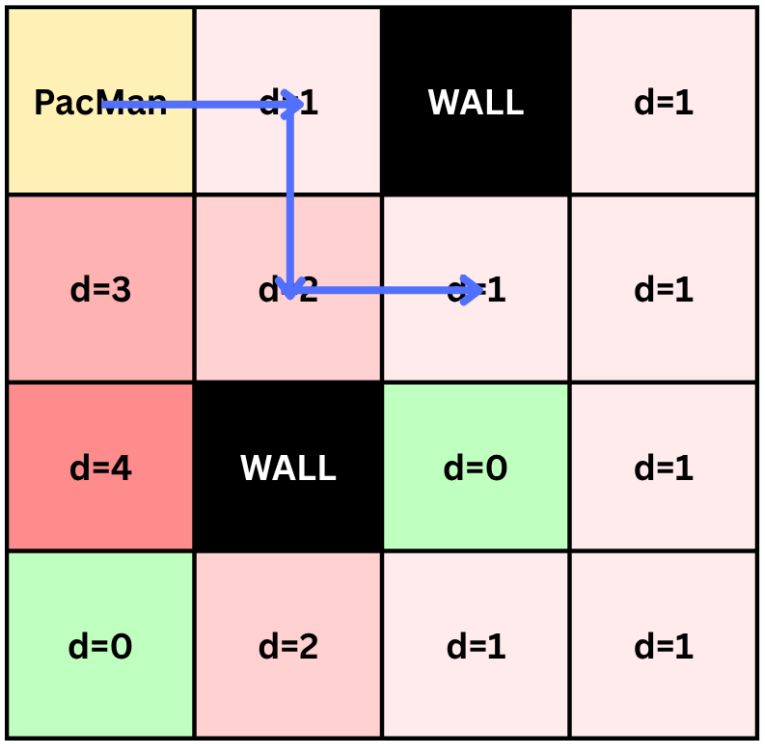
Cost = 1 + (d x 3)

PacMan AI: Ghost Avoidance (Weighted Dijkstra)



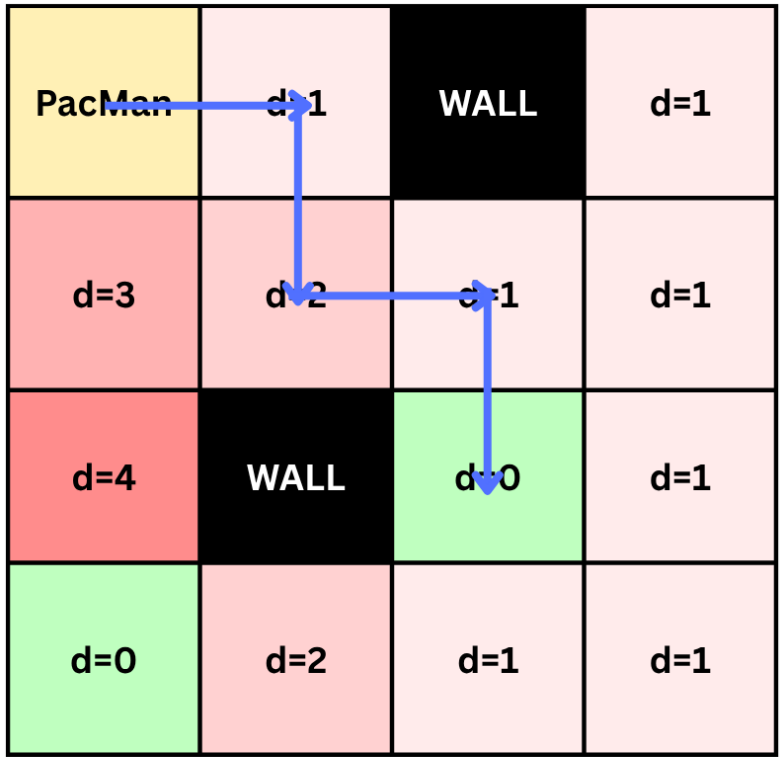
Cost = 1 + (d x 3)

PacMan AI: Ghost Avoidance (Weighted Dijkstra)



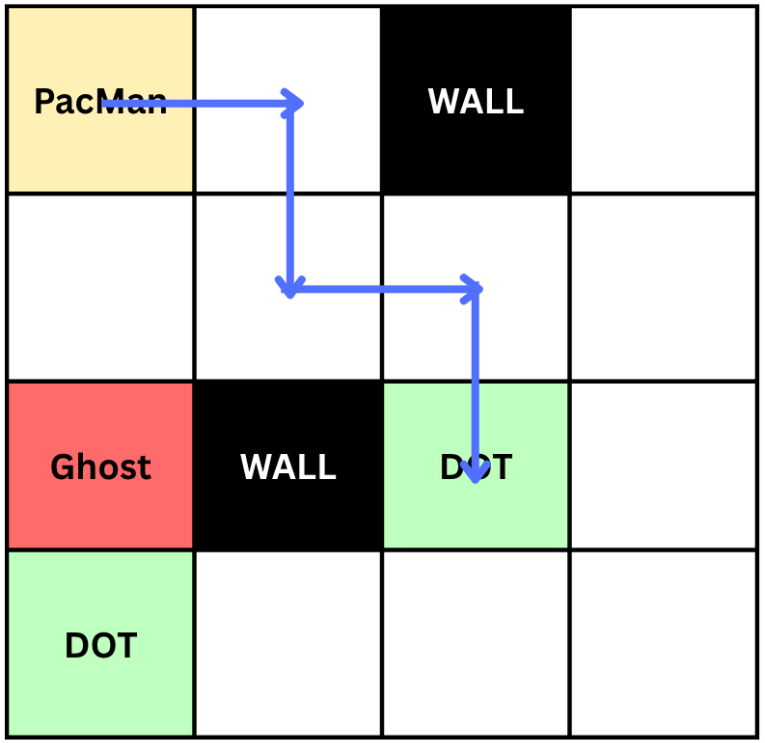
Cost = 1 + (d x 3)

PacMan AI: Ghost Avoidance (Weighted Dijkstra)

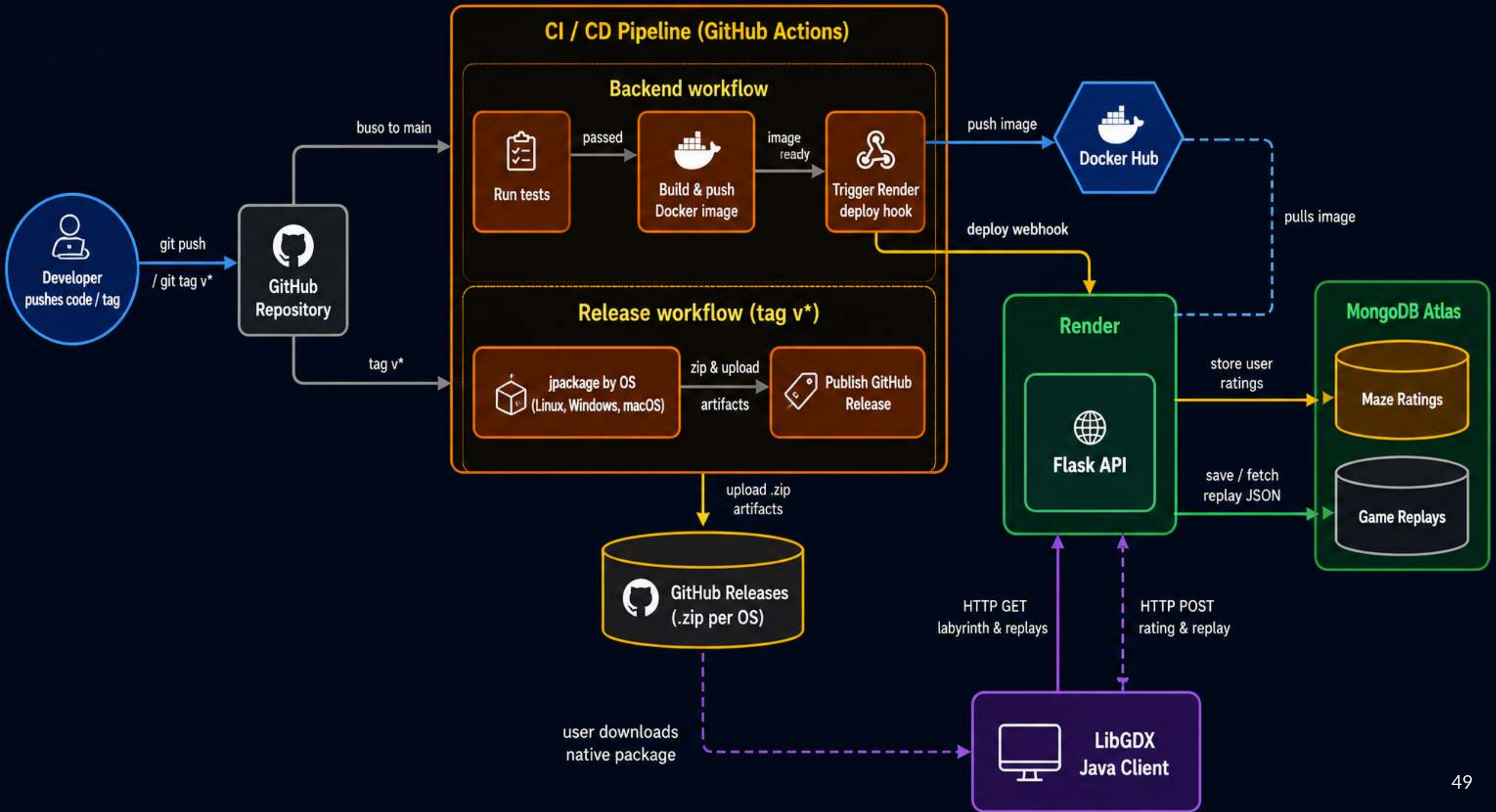


Cost = 1 + (d x 3)

PacMan AI: Ghost Avoidance (Weighted Dijkstra)



Cost = 16


CI / CD Pipeline (GitHub Actions)
Backend workflow
Run tests
passed
Build & push Docker image
image ready
Trigger Render deploy hook
Release workflow (tag v*)
jpackage by OS (Linux, Windows, macOS)
zip & upload artifacts
Publish GitHub Release
Developer pushes code / tag
git push / git tag v*
GitHub Repository
buso to main
tag v*
push image
Docker Hub
pulls image
deploy webhook
Render
Flask API
store user ratings
save / fetch replay JSON
MongoDB Atlas
Maze Ratings
Game Replays
upload .zip artifacts
GitHub Releases (.zip per OS)
HTTP GET labyrinth & replays
HTTP POST rating & replay
user downloads native package
LibGDX Java Client

Features



Full GUI

Desktop client developed using Java/LibGDX with Pac-Man rendering.



MongoDB Atlas Persistence

Utilized MongoDB Atlas for storing mazes, ratings, and replays.



Ghost AI

Introduced 4 ghosts with distinct individual behaviors and modes.



Difficulty Levels

Options for Classic (Greedy) and A* (Optimal Pathfinding) difficulty levels.



Player AI Bot

AI bot strategies including GREEDY and GHOST_AVOIDANCE.



Race Mode

Added a competitive race mode for players.



Replay System

Functionality to record and playback completed games.

Demo & Conclusion

Key Takeaways and Next Steps



Live Demo / Video Showcase

Interactive demonstration of the project's capabilities.